

Roboty mobilne

Instrukcja do zajęć laboratoryjnych

Ćwiczenie 4. Integracja symulatora robota mobilnego z portem szeregowym i oknem PyQt6

Wstęp

W ćwiczeniu 2 przygotowano testy automatyczne, fixture portu szeregowego oraz podstawy testów integracyjnych. W ćwiczeniu 3 przygotowano symulator robota różnicowego, w którym stan robota opisują x , y oraz θ , a komendy ruchu są wykonywane przez klasę `MotionCommand`.

Celem ćwiczenia 4 jest połączenie tych elementów w jeden przepływ pracy. Komendy prędkości kół mają zostać wprowadzone jako tekst, zapisane do pozornego portu szeregowego, odczytane, przeliczone na `MotionCommand`, wykonane przez symulator, zapisane do pliku CSV i pokazane na wykresie. Ostatnim elementem jest przygotowanie prostego okna PyQt6 z polem tekstowym i przyciskiem uruchamiającym eksperyment.

1. Wymagania wstępne

Przed rozpoczęciem ćwiczenia upewnij się, że w projekcie dostępne są elementy przygotowane wcześniej:

- `SerialPortFixture` oraz funkcje zapisu i odczytu z pozornego portu szeregowego,
- `RobotState`, `MotionCommand`, `wheel_speeds_to_velocity` oraz `simulate_commands`,
- funkcje zapisu trajektorii do CSV i rysowania wykresu 2D,
- folder `tests` oraz folder `testIntegration` z możliwością uruchamiania testów przez `pytest`.

Potrzebne biblioteki zainstaluj poleceniem:

```
pip install pytest matplotlib PyQt6
```

2. Zakres integracji

W tym ćwiczeniu należy zintegrować następujący przepływ danych:

```
tekstowe komendy prędkości kół
-> zapis do SerialPortFixture
-> odczyt z portu jako bytes
-> konwersja bytes -> str
-> parser komend
-> wheel_speeds_to_velocity
-> MotionCommand
-> simulate_commands
-> trajectory.csv, trajectory.png, summary.txt
-> test integracyjny
-> minimalne okno PyQt6
```

Interfejs PyQt6 nie powinien zawierać osobnej logiki symulacji. Ma jedynie pobrać tekst z pola tekstowego i uruchomić przygotowany wcześniej pipeline eksperymentalny.

3. Format komend sterowania

Każda linia komendy ma opisywać prędkość lewego koła, prędkość prawego koła oraz czas trwania ruchu:

$v_{left}, v_{right}, duration$

Przykładowy zestaw komend:

```
0.10,0.10,1.0
0.25,0.25,1.0
0.45,0.45,1.0
0.20,0.50,2.0
0.50,0.20,2.0
0.30,0.30,1.0
0.10,0.10,1.0
```

Nie należy wprowadzać osobnych komend tekstowych typu MOVE, ACCEL lub DECEL. Przyspieszanie oznacza kilka kolejnych komend z rosnącymi prędkościami kół. Spowalnianie oznacza kilka kolejnych komend z malejącymi prędkościami kół. Ruch po łuku lub ruch kołowy otrzymuje się wtedy, gdy prędkości kół są różne.

Warunek	Rodzaj ruchu	Znaczenie
$v_{left} = v_{right} > 0$	jazda prosto	robot jedzie bez zmiany orientacji
$v_{left} < v_{right}$	ruch po łuku	robot skręca w jedną stronę
$v_{left} > v_{right}$	ruch po łuku	robot skręca w drugą stronę
prędkości rosną w kolejnych liniach	przyspieszanie	robot zwiększa prędkość jazdy
prędkości maleją w kolejnych liniach	spowalnianie	robot zmniejsza prędkość jazdy

4. Zadanie 1 - parser komend z portu szeregowego

W pliku `command_parser.py` przygotuj funkcje, które zamienią tekst z portu szeregowego na listę obiektów `MotionCommand`.

```
def parse_wheel_command(line: str) -> tuple[float, float, float]:
    # line: "v_left,v_right,duration"
    # wynik: v_left, v_right, duration
    pass
```

```
def command_from_line(line: str, wheel_base: float) -> MotionCommand:
    # użyj parse_wheel_command oraz wheel_speeds_to_velocity
    pass
```

```
def commands_from_text(text: str, wheel_base: float) -> list[MotionCommand]:
    # podziel tekst na linie i pomiń puste wiersze
    pass
```

Do wykonania:

- obsłuż format $v_{left}, v_{right}, duration$,
- zgłoś `ValueError` dla niepełnej, pustej lub błędnej komendy,
- przelicz prędkości kół na v oraz ω ,
- zwróć listę `MotionCommand` dla wielu linii tekstu.

5. Zadanie 2 - uruchomienie eksperymentu z komend portu

W pliku `experiment_runner.py` przygotuj funkcję uruchamiającą cały przepływ od portu szeregowego do plików wynikowych.

```
def run_experiment_from_serial(
    serial_port,
    initial_state: RobotState,
    wheel_base: float,
    dt: float,
    output_dir: str,
) -> dict[str, float]:
    # 1. odczytaj tekst z portu
    # 2. zamień tekst na listę MotionCommand
    # 3. uruchom simulate_commands
    # 4. zapisz trajectory.csv i trajectory.png
    # 5. zapisz summary.txt
    pass
```

Do wykonania:

- wykorzystaj `SerialPortRead` z ćwiczenia 2,
- wykorzystaj `simulate_commands` z ćwiczenia 3,
- zapisz trajektorię i wykres do katalogu wynikowego,
- zwróć słownik z podstawowymi metrykami eksperymentu.

6. Zadanie 3 - metryki eksperymentu

W pliku `metrics.py` przygotuj krótkie podsumowanie wyników. Metryki mają pomóc porównać ruch prosty, przyspieszanie, spowalnianie oraz ruch po łuku.

```
summary = {
    "number_of_states": ...,
    "path_length": ...,
    "final_x": ...,
    "final_y": ...,
    "final_theta": ...,
    "min_command_speed": ...,
    "max_command_speed": ...,
}
```

Do wykonania:

- oblicz długość drogi na podstawie kolejnych punktów trajektorii,
- zapisz stan końcowy robota,
- wyznacz minimalną i maksymalną prędkość liniową z listy `MotionCommand`,
- sprawdź, czy przyspieszanie zwiększa `max_command_speed`, a spowalnianie zmniejsza końcowe wartości prędkości.

7. Zadanie 4 - program `main_experiment.py`

Program `main_experiment.py` powinien utworzyć pozorny port, zapisać do niego komendy, uruchomić eksperyment i wypisać metryki.

```
# przykład komend użytych w main_experiment.py
commands_text = """
0.10,0.10,1.0
0.25,0.25,1.0
0.45,0.45,1.0
0.20,0.50,2.0
0.50,0.20,2.0
0.30,0.30,1.0
```

0.10,0.10,1.0

"""

Do wykonania:

- zapisz commands_text do SerialPortFixture,
- uruchom run_experiment_from_serial,
- sprawdź, czy powstały pliki trajectory.csv, trajectory.png i summary.txt,
- otwórz trajectory.png i oceń, czy widoczny jest ruch prosty, ruch po łuku, przyspieszanie i spowalnianie.

8. Zadanie 5 - test integracyjny

Test integracyjny powinien sprawdzić cały przepływ: port szeregowy -> parser -> MotionCommand -> symulator -> pliki wynikowe.

```
def test_serial_to_simulator_pipeline(serial_port, tmp_path):
    # zapisz do portu kilka komend: przyspieszanie, ruch po łuku, spowalnianie
    # uruchom run_experiment_from_serial(..., output_dir=tmp_path)
    # sprawdź istnienie trajectory.csv, trajectory.png i summary.txt
    # sprawdź, czy path_length > 0
    # sprawdź, czy final_theta zmieniło się po ruchu po łuku
    pass
```

Do wykonania:

- użyj tmp_path, aby test nie zapisywał plików w głównym katalogu results,
- sprawdź istnienie trzech plików wynikowych,
- sprawdź, czy długość drogi jest większa od zera,
- sprawdź, czy scenariusz z różnymi prędkościami kół zmienia orientację robota.

9. Zadanie 6 - okno PyQt6

PyQt6 jest w tym ćwiczeniu częścią obowiązkową. Należy przygotować plik gui_app.py z gotowym oknem, polem tekstowym oraz przyciskiem. Kod interfejsu nie implementuje symulacji, lecz wywołuje funkcję run_experiment_from_serial przygotowaną w poprzednich zadaniach.

W pliku gui_app.py umieść następujący kod:

```
import sys
from pathlib import Path

from PyQt6.QtWidgets import (
    QApplication,
    QLabel,
    QPushButton,
    QTextEdit,
    QVBoxLayout,
    QWidget,
)

from robot_state import RobotState
from serial_fixture import SerialPortFixture
from serial_utils import SerialPortWrite
from experiment_runner import run_experiment_from_serial

class MainWindow(QWidget):
    def __init__(self) -> None:
        super().__init__()
        self.setWindowTitle("Symulator robota mobilnego")
        self.resize(700, 430)

        self.label = QLabel(
            "Komendy sterowania (v_left,v_right,duration):"
        )
```

```

self.commands_textbox = QTextEdit()
self.commands_textbox.setPlaceholderText(
    "Wpisz komendy, np.\n"
    "0.10,0.10,1.0\n"
    "0.25,0.25,1.0\n"
    "0.20,0.50,2.0"
)

self.button = QPushButton("Uruchom eksperyment")
self.status_label = QLabel("Status: oczekiwanie na komendy")

self.button.clicked.connect(self.run_experiment)

layout = QVBoxLayout()
layout.addWidget(self.label)
layout.addWidget(self.commands_textbox)
layout.addWidget(self.button)
layout.addWidget(self.status_label)
self.setLayout(layout)

def run_experiment(self) -> None:
    commands_text = self.commands_textbox.toPlainText().strip()

    if not commands_text:
        self.status_label.setText(
            "Status: wpisz co najmniej jedną komendę."
        )
        return

    serial_port = SerialPortFixture()
    SerialPortWrite(serial_port, commands_text + "\n")

    try:
        summary = run_experiment_from_serial(
            serial_port=serial_port,
            initial_state=RobotState(0.0, 0.0, 0.0),
            wheel_base=0.5,
            dt=0.05,
            output_dir=Path("results") / "gui_experiment",
        )
    except ValueError as error:
        self.status_label.setText(f"Status: błąd danych - {error}")
        return

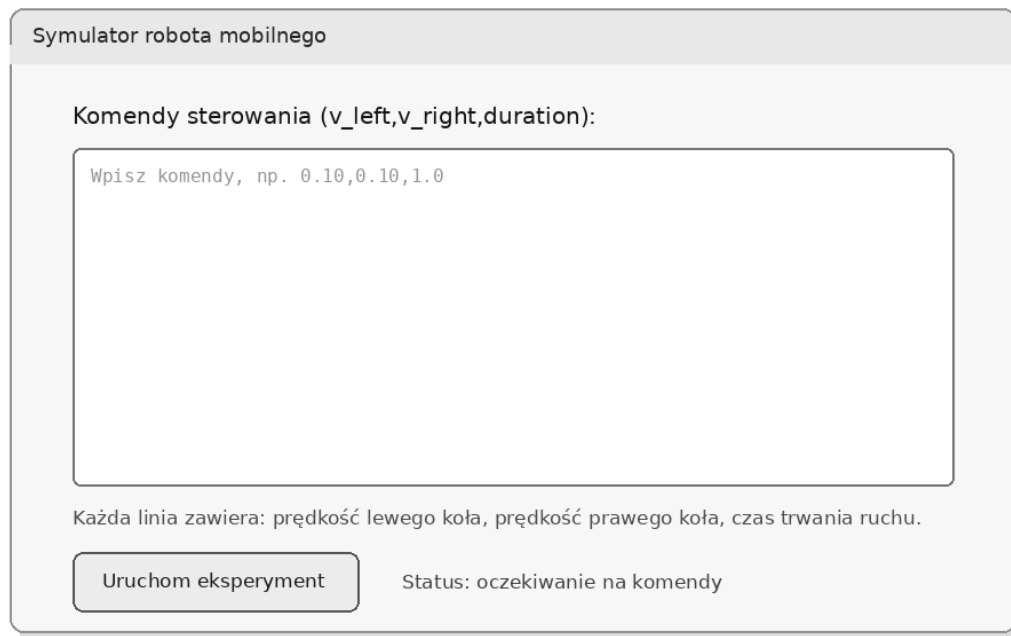
    self.status_label.setText(
        "Status: zapisano wyniki. "
        f"Długość drogi: {summary['path_length']:.3f} m"
    )

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```

Po uruchomieniu polecenia:

python gui_app.py
 powinno pojawić się okno podobne do przedstawionego poniżej.



Rys. 1. Okno PyQt6 po uruchomieniu programu, przed wpisaniem komend.

Do wykonania:

- utwórz plik `gui_app.py` i wklej podany kod,
- uruchom `python gui_app.py`,
- sprawdź, czy pojawia się okno z polem tekstowym i przyciskiem,
- wklej do textbxa komendy prędkości kół i kliknij Uruchom eksperyment,
- sprawdź, czy w `results/gui_experiment` zapisano wyniki eksperymentu.

10. Uruchomienie programu i testów

Po zakończeniu pracy uruchom:

```
pytest -v
python main_experiment.py
python gui_app.py
```

W katalogu wynikowym powinny znajdować się:

- `trajectory.csv`
- `trajectory.png`
- `summary.txt`
- zrzut ekranu okna PyQt6

Uwagi końcowe

Ćwiczenie 4 łączy testowanie, komunikację przez pozorny port szeregowy oraz symulator robota mobilnego.

Najważniejsze jest poprawne przekazanie danych między modułami. Jeżeli pipeline działa poprawnie z poziomu testu integracyjnego, okno PyQt6 powinno jedynie wywoływać ten sam sprawdzony mechanizm.